Recap for the whole lecture:

(#1): Hardware powers scale, and low-level details determine what scales or doesn't.

(#2): Current GPU based compute strongly encourage thinking about matmul + data movement

(#3): Thinking carefully about the GPU (coalescing, tiling, fusion) leads us to good performance.

Part #3. Performance kernel using Triton

kernel; C++ kernel; Triton kernel;

Lecture: high-level overview of GPUs and performance

This lecture: benchmarking/profiling + write kernels
↳ where time is spent?
↳ how long does it take?

write kernels: python using triton.
write kernels: C++ / ~~using~~ cuda.

5 ways to write a function: manual, pytorch, compiled, CUDA, Triton

GeLU (element-wise), softmax (row-wise), matmul (complex aggregation)

- key principle: organize computation to minize reads/write.
- key ideas: kernel fusion (warehouse/factory analogy), tiling (shared memory)
- Auto-compilers (Triton, torch.compile) will get better over time

execution model:

Thread: process individual index (i.e. f(i))
Thread block: (concurrent thread arrays): scheduled on a single SM
Grid: collection of thread blocks